Projet React débutant - Gestionnaire de tâches

But

Construire **un seul petit projet**, étape par étape :

```
Afficher → ajouter → filtrer → charger depuis une API → modifier → supprimer
```

Ne recommencez pas le projet à chaque exercice. Gardez le code précédent et ajoutez seulement ce qui est demandé.

Signification des consignes

- **CRÉER** : ajouter un nouveau fichier.
- **REMPLACER** : effacer le contenu du fichier et mettre le nouveau code.
- **AJOUTER** : garder le contenu actuel et insérer quelques lignes.
- **MODIFIER** : remplacer seulement la partie indiquée.

---

0. Préparer le projet

ACTION : CRÉER un nouveau projet React.

Dans PowerShell :

```
npm create vite@latest react-tasks -- --template react-ts
cd react-tasks
npm install
npm run dev
```

Ouvrez l'adresse affichée, normalement `http://localhost:5173` .

Tous les exercices React modifient le même fichier : `src/App.tsx` .

---

1. JavaScript : comprendre les données

ACTION : CRÉER `javascript-basics.js` à la racine de `react-tasks` .

```
const tasks = [
  { id: 1, title: "Lire le rapport", done: true },
  { id: 2, title: "Vérifier les alertes", done: false },
];
```

Ajoutez du code pour :

1. afficher le titre de la première tâche ;
2. créer `remainingTasks` avec seulement les tâches non terminées ;
3. afficher le nombre de tâches non terminées.

À expliquer : quelle est la différence entre un tableau et un objet ?

---

2. Premier composant React

ACTION : REMPLACER tout le contenu de `src/App.tsx` .

Créez :

- `TaskHeader` , qui affiche `Gestionnaire de tâches` ;
- `App` , qui affiche `<TaskHeader />` dans `<main>` .

```
function TaskHeader() {
  // Retourner un titre h1.
}

export default function App() {
  // Afficher TaskHeader dans main.
}
```

À expliquer : qu'est-ce qu'un composant ? Pourquoi son nom commence-t-il par une majuscule ?

---

3. Props et état

ACTION : AJOUTER le code dans `src/App.tsx` . Ne supprimez pas `TaskHeader` .

Écrivez ce type **une seule fois** :

```
type Task = {
  id: number;
  title: string;
  done: boolean;
};
```

Ajoutez ensuite :

1. un composant `TaskItem` qui reçoit une prop `task` ;
2. l'état `tasks` dans `App` avec deux tâches initiales ;
3. un `map` qui affiche un `TaskItem` par tâche.

Le type `Task` sera réutilisé jusqu'à la fin. **Ne le réécrivez plus.**

À expliquer : une prop vient-elle du parent ou de l'enfant ? À quoi sert `useState` ?

---

4. Ajouter une tâche

ACTION : AJOUTER `TaskForm` dans `src/App.tsx` , au-dessus de `App` .

Le formulaire doit contenir :

- un état `title` ;
- un champ contrôlé ;
- un bouton `Ajouter` ;
- une prop `onAdd(title)` .

ACTION : AJOUTER `addTask` dans `App` .

`addTask` doit ajouter ceci à l'état `tasks` :

```
{ id: Date.now(), title, done: false }
```

ACTION : AJOUTER le formulaire dans le JSX de `App` .

```
<TaskForm onAdd={addTask} />
```

Testez un titre vide et un titre valide.

À expliquer : à quoi servent `onChange`, `onSubmit` et `event.preventDefault()` ?

---

5. Terminer et filtrer les tâches

ACTION : MODIFIER `TaskItem` dans `src/App.tsx`.

Ajoutez une prop `onToggle`. Un clic doit transmettre `task.id` au parent.

ACTION : AJOUTER dans `App`.

1. `toggleTask(taskId)`, qui inverse `done` avec `map` ;
2. l'état `remainingOnly` ;
3. `visibleTasks`, calculé avec `filter` ;
4. une case à cocher pour activer le filtre.

ACTION : MODIFIER la liste.

Utilisez `visibleTasks.map(...)` au lieu de `tasks.map(...)`. Affichez `Aucune tâche` si la liste est vide.

À expliquer : quelle est la différence entre `map` et `filter` ? Pourquoi utiliser `task.id` comme `key` ?

---

6. Utiliser `useEffect`

ACTION : MODIFIER l'import de `src/App.tsx`.

Ajoutez `useEffect` à l'import React.

ACTION : AJOUTER dans `App`, avant le `return`.

Calculez le nombre de tâches non terminées, puis affichez-le dans le titre de l'onglet :

```
2 tâche(s) restante(s)
```

L'effet doit dépendre de `remainingCount`.

À expliquer : pourquoi `document.title` est-il modifié dans `useEffect` ?

---

7. Créer la petite API

ACTION : CRÉER le dossier `api` à la racine de `react-tasks` .

ACTION : CRÉER `api/main.py` .

Dans PowerShell :

```
py -m venv .venv
.venv\Scripts\Activate.ps1
python -m pip install fastapi "uvicorn[standard]"
```

Si la commande `py` n'existe pas, utilisez plutôt :

```
uv venv --python 3.12 .venv
.venv\Scripts\Activate.ps1
uv pip install fastapi "uvicorn[standard]"
```

Dans `api/main.py` , créez :

1. `app = FastAPI()` ;
2. une liste Python avec les deux mêmes tâches ;
3. `GET /tasks` , qui retourne la liste ;
4. CORS pour `http://localhost:5173` .

Lancez l'API dans un **deuxième terminal** :

```
python -m uvicorn api.main:app --reload
```

Testez `GET /tasks` dans `http://127.0.0.1:8000/docs` .

À expliquer : React et FastAPI fonctionnent-ils dans le même programme ?

---

8. Charger les tâches de l'API

ACTION : MODIFIER l'état `tasks` dans `src/App.tsx` .

Commencez avec un tableau vide :

```
const [tasks, setTasks] = useState<Task[]>([]);
```

ACTION : AJOUTER en haut de `src/App.tsx` .

```
const API_URL = "http://127.0.0.1:8000/tasks";
```

ACTION : AJOUTER dans `App` .

1. les états `loading` et `error` ;
2. un `useEffect` qui appelle `fetch(API_URL)` ;
3. `setTasks(data)` après `response.json()` ;
4. `Chargement...` ou un message d'erreur dans le JSX.

Gardez le formulaire, le filtre et la liste déjà créés.

À expliquer : que sont HTTP et JSON ? Pourquoi `fetch` utilise-t-il `await` ?

---

9. Terminer le CRUD

CRUD signifie : créer, lire, modifier et supprimer.

Action	Méthode
Créer	`POST`
Lire	`GET`
Modifier	`PATCH`
Supprimer	`DELETE`

API

ACTION : AJOUTER dans `api/main.py` .

- `POST /tasks` ;
- `PATCH /tasks/{task_id}` ;
- `DELETE /tasks/{task_id}` .

Ne recréez pas le fichier. Gardez `GET /tasks`.

React

ACTION : MODIFIER les fonctions existantes dans `src/App.tsx`.

- `addTask` doit utiliser `POST` ;
- `toggleTask` doit utiliser `PATCH`.

ACTION : AJOUTER `deleteTask` et un bouton `Supprimer`.

ACTION : MODIFIER `TaskItem`.

Placez les boutons `Changer le statut` et `Supprimer` dans un même `<li>`. Ne placez pas un bouton à l'intérieur d'un autre bouton.

ACTION : MODIFIER l'utilisation de `<TaskItem />` dans la liste.

Transmettez aussi `onDelete={deleteTask}`.

Mettez l'état React à jour seulement après une réponse réussie de l'API.

Vérification finale

- [ ] Je sais dans quel fichier j'écris chaque partie.
- [ ] Je peux expliquer composant, prop et état.
- [ ] Je peux ajouter et filtrer une tâche.
- [ ] Je comprends `useEffect` et `fetch`.
- [ ] Je peux associer CRUD à `POST`, `GET`, `PATCH` et `DELETE`.
- [ ] Je peux expliquer ce trajet :

```
React → requête HTTP → FastAPI → réponse JSON → setTasks → affichage
```